

C Programming

4_bitwise_ops

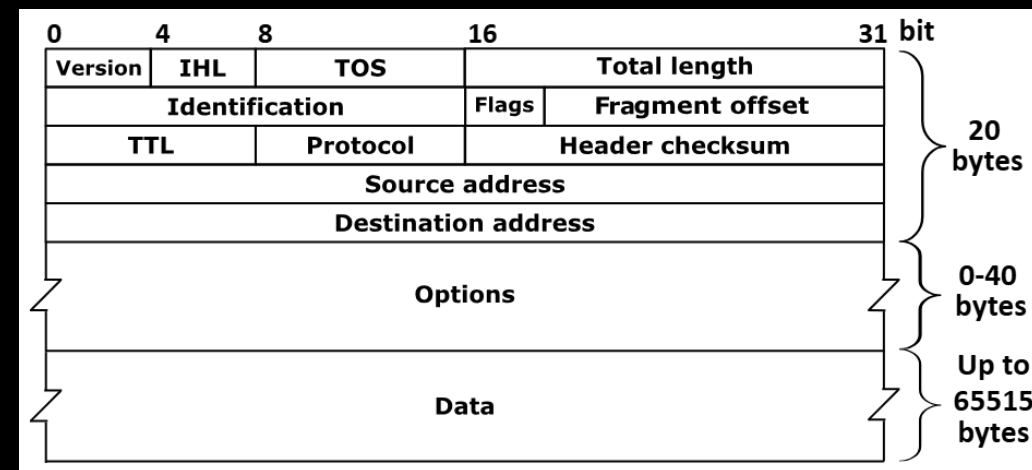
Зачем?

Битовые операции:

- Позволяют работать с отдельными битами
- Работают быстрее некоторых арифметических операций

Например:

- Картинка с котиком из интернета (метаинформация)
- **IP пакет**, который доставил нам эту картинку, он также использует **отдельные биты в своем заголовке**
- Различные алгоритмы шифрования



printf bin?

```
char dec = 123;  
char hex = 0x7b; // or 0x7B  
char oct = 0173;  
char bin = 0b1111011;
```

```
printf("%d == %d == %d == %d", dec, hex, oct, bin);  
printf("%d == %x == %o == ?", dec, hex, oct);
```

Bitwise Operators

&	AND (И)
	OR (ИЛИ)
>>, <<	Shift (Сдвиг)
~	NOT (Отрицание, НЕ)
^	XOR (Искл. ИЛИ)

Не путать: Один символ `&` - побитовое И, два символа `&&` - логическое И, тоже самое с ИЛИ

AND (∩) “&”

a	b	&
0	0	0
0	1	0
1	0	0
1	1	1

a = 3, b = 7

a & b = 3

3 = 00000011

7 = 00000111

&= 00000011

$a \& b = ?$

$a = 17, b = 5$

$$a \& b = ?$$

$$a = 17, b = 5$$

$$a \& b = 1$$

OR (ИЛИ) “|”

a	b	
0	0	0
0	1	1
1	0	1
1	1	1

a = 3, b = 7

a | b = 7

3 = 00000011

7 = 00000111

| = 00000111

$$a \mid b = ?$$

$$a = 17, b = 6$$

$$a \mid b = ?$$

$$a = 17, b = 6$$

$$a \mid b = 23$$

XOR (Исключающее ИЛИ) “^”

a	b	^
0	0	0
0	1	1
1	0	1
1	1	0

$$a = 3, b = 7$$

$$a \wedge b = 4$$

$$3 = 00000011$$

$$7 = 00000111$$

$$| = 00000100$$

$$a \wedge b = ?$$

$$a = 17, b = 8$$

$$a \wedge b = ?$$

$$a = 17, b = 8$$

$$a \wedge b = 25$$

NOT (Отрицание) $\sim$

a	$\sim$
0	1
1	0

$$a = 3$$

$$\sim a = -4 \text{ (252)}$$

$$3 = 00000011$$

$$\sim = 11111100$$

<https://www.rapidtables.com/convert/number/binary-to-decimal.html?x=11111100>

$\sim a?$

$a = -5$ (0b11111011)

$\sim a?$

$a = -5$ (0b11111011)

$\sim a = 4$

Shift (Сдвиг) “>>”, “<<”

```
char a = 3;
```

```
a = a << 1;
```

```
char b = a >> 2;
```

Сдвиг работает только для **целых чисел**

Left shift (Сдвиг влево) “<<”

$a = 3 = 00000011$

$a \ll 1 = 00000110$

$a = 3 = 00000011$

$a \ll 5 = 01100000$

$a = 3 = 00000011$

$a \ll 7 = 10000000$

Right shift (Сдвиг вправо) ">>"

`a = 3    = 00000011`

`a >> 1 = 00000001`

`char a = -128; // = 10000000`

`a = a >> 1;     // = 11000000 = -64`

$(a \gg 24) \ll 8 = ?$

`int a = 0xBAADF00D`

$$(a \gg 24) \ll 8 = ?$$

```
int a = 0xBAADF00D
```

1. 0x000000BA
2. 0x0000BA00

Mult by 2

Можно заметить, что при **сдвиге числа влево** мы его **умножаем на 2** в степени **количества сдвинутых бит**. Простыми словами: сдвигая влево на **1 бит** умножаем на **2**, сдвигая на **2 бита** умножаем на **4**, на **3 бита** умножаем на **8** и т.д..

$$a = 3 = 00000011$$

$$a \ll 4 = 00110000 = 3 * 2^4 = 48$$

Div by 2

Сдвигая вправо на 1 бит - делим на 2 и т.д..

$$a = 17 = 00010001$$

$$a \gg 1 = 00001000 = 17 / 2 = 8$$

Extract byte

```
unsigned int a = 0x00C0FFEE;  
unsigned char c;  
c = (a & 0x000000FF); // тоже что и (a & 0xFF)  
printf("first byte of a = %x\n", c); // c == 0xEE  
  
c = ((a >> 8) & 0x000000FF);  
printf("second byte of a = %x\n", c); // c == 0xFF
```

Extract bit

$(x \gg i) \& 1$

где

- x - исходное число,
- i - номер искомого бита

```
unsigned char a = 17;  
unsigned char b = (a >> 5) & 1; // 5 бит с нуля (6 бит)
```

sizeof

sizeof() - дословно размер чего-либо что в скобках, это **оператор**, знающий размеры типов данных и переменных в **байтах**

```
char a;  
sizeof(a); // = 1 байт  
sizeof(int); // = 4 байта (вероятнее всего)
```

print binary

```
char dec = 123;  
char hex = 0x7b; // or 0x7B  
char oct = 0173;  
char bin = 0b01111011;  
  
printf("%d == %d == %d == %d\n", dec, hex, oct, bin);  
printf("%d == 0x%x == 0%o == 0b", dec, hex, oct);
```

Github

<https://github.com/kruffka/C-Programming>

